Module Guide for Industrial Plant Designer

Team 18, Chemware Engineering
Kishor Pandya
Dhrumil Pandya
Madhu Sivapragasam
Rasik Pokharel

April 3, 2025

1 Revision History

Date	Version	Notes
Jan 5th 2025	1.0	Madhu Sivapragasam: Added auth and restore module definitions
Jan 6th 2025	1.0	Dhrumil Pandya: Added save and system data module definitions
Jan 6th 2025	1.0	Madhu Sivapragasam: Added timeline and module hierarchy sections
Jan 6th 2025	1.1	Kishor Pandya: Added user data and dashboard module definitions
Jan 8th 2025	1.1	Kishor Pandya: Added user interface and connection between req and design sections
Jan 13th 2025	1.2	Dhrumil Pandya: Added Design of Communication Protocols section and use hierarchy between modules section
Jan 13th 2025	1.2	Rasik Pokharel: Added canvas module and anticipated changes
Jan 16th 2025	1.3	Madhu Sivapragasam: Add traceability matrix between requirements and modules
Jan 16th 2025	1.3	Madhu Sivapragasam: Add Simulation module
Jan 16th 2025	1.4	Rasik Pokharel: Add traceability matrix between AC modules
April 3rd 2025	2.0	Madhu Sivapragasam: Small changes for rev 1

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
Industrial Plant Designer	Name of the factory simulation software that is being developed
UC	Unlikely Change

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 3

5 Module Hierarchy **4**

6 Connection Between Requirements and Design **5**

7 Module Decomposition **5**
7.1 Hardware Hiding Modules (M1) 6
7.2 Behaviour-Hiding Module 6
7.2.1 Authentication Module (M2) 6
7.2.2 Restore Module (M4) 6
7.2.3 Save Module (M3) 7
7.2.4 System Data Module (M6) 7
7.2.5 User Data Module (M7) 7
7.2.6 Dashboard Module (M8) 7
7.2.7 Canvas Module (M5) 8
7.3 Software Decision Module 8
7.3.1 Simulation Module (M9) 8

8 Traceability Matrix **8**

9 Use Hierarchy Between Modules **11**

10 User Interfaces **11**

11 Design of Communication Protocols **13**
11.1 Request–Response Model 13
11.2 Authentication 13
11.3 Payload and Data Serialization 13
11.4 Status Code and Error Handling 14
11.5 Example Request and Response Structures 14
11.5.1 GET Request Example (Fetch Domain Data) 14
11.5.2 POST Request Example (Save a New Diagram) 15
11.5.3 PUT Request Example (Update Existing Diagram) 15

11.5.4 DELETE Request Example (Remove a Refresh Token) 16

12 Timeline **16**

13 Appendix **17**

List of Tables

1 Module Hierarchy 5
2 Trace Between Requirements and Modules 10
3 Trace Between Anticipated Changes and Modules 11
4 Development Timelome 17

List of Figures

1 Use hierarchy among modules 11
2 Login Page 17
3 Sign-up Page 18
4 Dashboard Page 18
5 Dashboard Page, when create is clicked 19
6 Canvas Page 19
7 Canvas Page when in use 20

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (Parnas et al., 1984). We advocate a decomposition based on the principle of information hiding (Parnas, 1972). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is valuable in SC, where modifications are frequent, especially during initial development as the solution space is explored.

Our design follows the rules layed out by Parnas et al. (1984), as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (Parnas et al., 1984). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document can be a guide for a new project member to easily understand the overall structure and quickly find the relevant modules they are searching for.
- Maintainers: The hierarchical structure of the module guide improves the maintainers' understanding when they need to make changes to the system. It is important for a maintainer to update the relevant sections of the document after changes have been made.
- Designers: Once the module guide has been written, it can be used to check for consistency, feasibility, and flexibility. Designers can verify the system in various ways, such as consistency among modules, feasibility of the decomposition, and flexibility of the design.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes of the software requirements. Section 5 summarizes the module decomposition that was constructed according to the likely changes. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relation between anticipated changes and the modules. Section 9 describes the use relation between modules.

4 Anticipated and Unlikely Changes

This section lists possible changes to the system. According to the likeliness of the change, the possible changes are classified into two categories. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

These anticipated and unlikely changes still apply even after the completion of revision 1 since the project is being passed over to another team. For that new teams work they will still need to consider the reason and impact for these changes and so rather than remove the changes we have chosen to continue listing them below.

4.1 Anticipated Changes

Anticipated changes are the source of the information that is to be hidden inside the modules. Ideally, changing one of the anticipated changes will only require changing the one module that hides the associated decision. The approach adapted here is called design for change.

AC1: User Interface Updates:

- **Reason:** Users may request a more intuitive or visually appealing interface. These changes are anticipated to resemble existing market solutions.
- **Impact:** Modifications to the Canvas Module to enhance features or improve visualization.

AC2: Authentication Mechanism Enhancements:

- **Reason:** As security needs evolve, requiring stricter authentication methods stricter methods may become required. Multi-factored authentication is anticipated.
- **Impact:** Updates to the Authentication Module to support new security frameworks and facilitate more users.

AC3: Integration with New Data Sources:

- **Reason:** As the system functionality is expanded to include more data sources for simulation and visualization new data integrations are expected.
- **Impact:** Changes to the System Data Module or User Data Module to handle new data.

AC4: Scaling Requirements:

- **Reason:** As the system increases, the performance of the system does as well. Especially given the expected increase in concurrent users, performance optimization will be a key piece in scaling the system.

- **Impact:** Optimization of all the data modules M3-M7 and M9 to handle larger datasets efficiently.

AC5: New Features or Functionalities:

- **Reason:** As the implementation timeline gets closer, the users of the system push for features and functionalities that they didn't initially anticipate.
- **Impact:** Updates to all the behavior-hiding modules may be required to incorporate new features.

AC6: Dependency Updates:

- **Reason:** Any dependencies such as react libraries that were used in the creation of existing modules are subject to change.
- **Impact:** Canvas, Save and Restore, Dashboard could all be impacted.

AC7: System-Wide Performance Improvements:

- **Reason:** The need to reduce latency and optimize system response times, especially optimizing simulation script interfacing.
- **Impact:** Refactoring of modules such as System Data Module or Save Module to optimize routing and database queries.

AC8: Hardware of the system:

- **Reason:** Changes in hosting decisions could change the hardware and underlying OS of the systems involved.
- **Impact:** M1 Hardware hiding modules will be impacted by this change.

4.2 Unlikely Changes

The module design should be as general as possible. However, a general system is more complex. Sometimes this complexity is not necessary. Fixing some design decisions at the system architecture stage can simplify the software design. If these decisions should later need to be changed, then many parts of the design will potentially need to be modified. Hence, it is not intended that these decisions will be changed.

UC1: Core Data Structure Changes:

- **Reason:** The system is built on specific data structures optimized for its operations that will not be subject to change. These changes are not likely to occur as they've been thoroughly finalized.
- **Impact:** Changing these core data structures would require significant architectural changes.

UC2: Removal of any existing features:

- **Reason:** The system is being built from the base up, all existing features will remain.
- **Impact:** Features are being built on top of each other, removing any existing core feature could be breaking change.

UC3: Removal of Modular Design:

- **Reason:** The modular structure is fundamental to the system's scalability and maintainability.
- **Impact:** Transitioning to a monolithic architecture would require significant rework and is unlikely given the benefits of modular design.

UC4: System Language Change:

- **Reason:** The system language has been thoroughly thought about and decided upon.
- **Impact:** Changing the primary language would require rewriting all code, libraries, and integrations.

UC5: Elimination of API Support:

- **Reason:** API integrations are crucial for the system to connect with external services and data sources.
- **Impact:** Removing API support would hinder functionality and require redesigning the System Data Module and User data module.

5 Module Hierarchy

This section provides an overview of the module design. Modules are summarized in a hierarchy decomposed by secrets in Table 1. The modules listed below, which are leaves in the hierarchy tree, are the modules that will actually be implemented.

M1: Hardware-Hiding Module

M2: Authentication Module

M3: Save Module

M4: Restore Module

M5: Canvas Module

M6: System Data Module

M7: User Data Module

M8: Dashboard Module

M9: Simulation Module

Level 1	Level 2
Hardware-Hiding Module	M1
Behaviour-Hiding Module	M2 M3 M4 M5 M6 M7 M8
Software Decision Module	M9

Table 1: Module Hierarchy

6 Connection Between Requirements and Design

The design of the system is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by Parnas et al. (1984). The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *Industrial Plant Designer* means the module will be implemented by the Industrial Plant Designer software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules (M1)

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Authentication Module (M2)

Secrets: The methods for handling authentication of users into the application

Services: Enables users to register an account, login to their account and validates all requests made by the user to the API modules

Implemented By: [Industrial Plant Designer]

Type of Module: [Abstract Object]

7.2.2 Restore Module (M4)

Secrets: The method for recovering previously built diagrams

Services: Provides a list of previously developed diagrams and enables diagram data to be loaded into the canvas module

Implemented By: [Industrial Plant Designer]

Type of Module: [Library]

7.2.3 Save Module (M3)

Secrets: The process of storing and updating diagrams (including domain snapshots) into the user database.

Services: Provides creation and update endpoints for saving diagrams, canvas data, and domain snapshots.

Implemented By: [Industrial Plant Designer]

Type of Module: [Library]

7.2.4 System Data Module (M6)

Secrets: The underlying routing and request-handling logic to interact with the system data.

Services: Exposes GET HTTP routes for domain data retrieval.

Implemented By: [Industrial Plant Designer]

Type of Module: [Abstract Object]

7.2.5 User Data Module (M7)

Secrets: The underlying routing and request-handling logic to interact with the user data.

Services: Exposes GET, POST, and PUT HTTP routes for user data manipulation.

Implemented By: [Industrial Plant Designer]

Type of Module: [Abstract Object]

7.2.6 Dashboard Module (M8)

Secrets: The methods for handling users choose between creating new canvas or loading an existing one.

Services: Provides a list of previously developed diagrams and/or enables a new canvas module to be created.

Implemented By: [Industrial Plant Designer]

Type of Module: [Library]

7.2.7 Canvas Module (M5)

Secrets: The underlying logic for data-driven nodes and edges.

Services: Displays the nodes and edges to the screen displaying data provided by various data modules. Allows users to interact with system data in each node and edge. Users can drag and drop, reshape, move the canvas, and perform other manipulations. Implements node and edge core functionalities using the React library "React Flow.

Implemented By: [Industrial Plant Designer]

Type of Module: [Abstract Object]

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Simulation Module (M9)

Secrets: The model that computes the outputs of the factory system based off of the inputs

Services: Computes output data for the factory using an internal algorithm and the provided inputs.

Implemented By: Professor Mahelec (capstone sponsor)

Type of Module: [Library]

8 Traceability Matrix

This section shows two traceability matrices: between the modules and the requirements and between the modules and the anticipated changes.

Req.	Modules
FR1	M5, M6
FR2	M5, M6

Continued on next page...

Req.	Modules
FR3	M9
FR4	M5, M3
FR5	M2, M3. M7
FR6	M2, M4, M7, M8
FR7	M5
FR8	M4, M8
FR9	M3, M5, M7
FR10	M5
LAFAR1	M5
LAFAR2	M5, M8
LAFAR3	M5
LAFSR1	M5, M8
LAFSR2	M5, M8
LAFSR3	M5, M8
UHE1	M5, M8
UHE2	M5, M8
UHPI1	M5, M8
UHL1	All modules
UHL2	M5, M8
UHU1	M6, M5, M8
UHA1	M1
PS1	M5, M8
PS2	M3
PS3	M4
PS4	M9
PPA1	M9, M5
PPA2	M5
PRF1	M9
PRF2	M5, M6, M3
PC1	M7, M5, M3, M4
PSE1	All modules
PL1	All modules

Continued on next page...

Req.	Modules
OEEP1	M1
OEEP2	All modules
OEEP3	M1
OEWE1	M2, M3, M4
OEWE2	M7, M6
OEIA1	M4, M8
OEIA2	M9
OEP1	All modules
OER1	All modules
MSM1	All modules
MSM2	All modules
MSS1	All modules
MSA1	All modules
MSA2	All modules
~~SEC-ACC1~~	~~M2~~
SEC-INT1	M5, M3
SEC-INT2	M2, M7, M6
SEC-PRI1	M2, M7
SEC-AUD1	M2, M7
SEC-IMM1	All modules
CR1	All modules
LR1	All modules
LR2	All modules

Table 2: Trace Between Requirements and Modules

AC	Modules
AC1	M5
AC2	M2
AC3	M6, M7
AC4	M6, M7,M3, M4, M9
AC5	M2,M6, M7,M3, M4,
AC6	M5, M3, M4, M3, M8
AC7	M6, M3
AC8	M1

Table 3: Trace Between Anticipated Changes and Modules

9 Use Hierarchy Between Modules

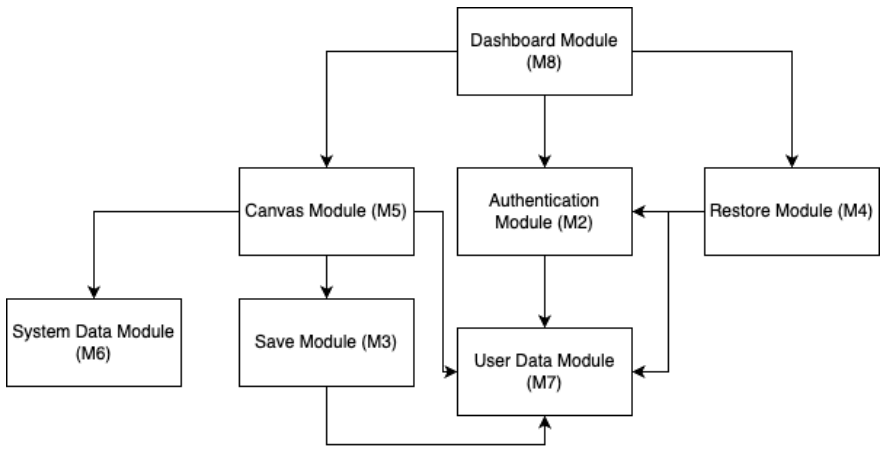

Figure 1: Use hierarchy among modules

10 User Interfaces

The user interface of the app includes multiple screens, whose screenshots have been added in the appendix. You can also view the multiple screens on Figma through this link.

The user interface is designed to allow users ease of use and functionality. The authentication pages all have a simple UI easy for anyone to understand. The canvas page is designed to help users build plant models and be able to visualize the model and all its data in one place.

Layout and Structure

The UI follows a structured layout with clear divisions, likely employing grid-based design principles. The interface emphasizes simplicity and usability, ensuring content and interactive elements are accessible.

Header or Navigation Bar: The top of the page may include navigation controls, such as menus or titles.

Components

- **Buttons:** The interface includes clickable buttons, styled with rounded corners, clear labels, and hover effects. Buttons are functional for actions like submitting forms, navigating between sections, or initiating specific tasks.
- **Input Fields:** Text input fields are prominently displayed, designed for user interaction. Labels or placeholders guide users to enter the required data.
- **Interactive Widgets:** Dropdowns, checkboxes, or other selection tools are present to allow users to make choices. Sliders or toggles may allow for dynamic adjustments.
- **Modals or Pop-ups:** If the interface uses modals, they highlight additional information or options while keeping the background slightly dimmed.
- **Icons:** Relevant icons visually complement buttons or other actions, making the interface intuitive.

Visual Design

- **Color Scheme:** A clean and professional palette is used, with contrasts for primary actions (e.g., a bright accent color for buttons). Background colors are neutral, ensuring readability.
- **Typography:** Fonts are consistent, with distinct styles for headings, subheadings, and body text. Adequate spacing is maintained to avoid clutter.
- **Spacing and Padding:** Generous padding and margins separate sections, enhancing readability and reducing cognitive load.

Navigation and Interaction

- **Flow:** The design supports a step-by-step or task-oriented flow, guiding users through actions intuitively. Navigation elements (e.g., a sidebar or breadcrumbs) ensure seamless movement across sections.
- **Responsiveness:** The layout is adaptive, ensuring usability on multiple devices, though this can only be confirmed through testing.

Purpose and Context

The UI serves specific functions, such as:

- Collecting user input or data.
- Displaying analytics, dashboards, or summaries.
- Providing actionable controls for workflows or applications.

11 Design of Communication Protocols

Our system implements a standard RESTful communication protocol over HTTP for all client–server interactions. Each module providing services (e.g., saving diagrams, restoring diagrams, retrieving system/domain data, user account creation, etc.) exposes one or more HTTP endpoints. All request and response bodies are encoded in JSON.

11.1 Request–Response Model

- The frontend (Canvas and Dashboard modules in the browser) sends requests (e.g., `GET`, `POST`, `PUT`, `DELETE`) via Axios to the backend
- The server responds to each request with a JSON response indicating success or error states

11.2 Authentication

- JWT (JSON Web Tokens) are used for stateless authentication
- Clients get a `refreshToken` upon successful login
- All subsequent API requests include the `accessToken` in the `Authorization` header (`Bearer <token>`) to authenticate
- The Authentication Module (`mAuth`) checks token validity and only allows authorized users to access or mutate the data

11.3 Payload and Data Serialization

- Diagram data and domain snapshots are serialized as "stringified" JSON when sent to or stored in the database
- User data (email, password hashes, profile information) is similarly sent over HTTPS as JSON

11.4 Status Code and Error Handling

- Standard HTTP status codes are used for different types of responses:
  - `200 OK`
  - `201 Created`
  - `400 Bad Request`
  - `401 Unauthorized`
  - `403 Forbidden`
  - `404 Not Found`
  - `500 Internal Server Error`
- Errors are returned in JSON format:

```
{
    "message": "Invalid credentials",
    "details": { ... }
}
```

11.5 Example Request and Response Structures

In this section, we have outlined the example of a few requests and responses that can be made to the backend server.

11.5.1 GET Request Example (Fetch Domain Data)

```
Request:
GET /api/data/domain/{domainId}
Authorization: Bearer <accessToken>

Response (200 OK):
{
  "id": "d1234",
  "name": "ChemicalDomain",
  "models": [...],
  "streams": [...],
  "colors": [...]
}
```

The server returns a 404 error if the requested domain does not exist, e.g.:

```
Response (404 Not Found):
{
  "message": "Domain not found"
}
```

11.5.2 POST Request Example (Save a New Diagram)

```
Request:
POST /api/data/diagrams
Authorization: Bearer <accessToken>
Content-Type: application/json

Body:
{
  "name": "MyFirstDiagram",
  "canvas": "{ \"nodes\": [...], \"edges\": [...], ... }",
  "parameters": "{ ... }",
  "domainId": "d1234",
  "snapshotData": { ... }
}

Response (201 Created):
{
  "message": "Diagram created successfully",
  "diagram": {
    "id": "diag5678",
    "name": "MyFirstDiagram",
    "canvas": "...",
    "parameters": "...",
    "snapshotId": "snap9999"
  }
}
```

If missing required fields, the server returns a 400 error, e.g.:

```
Response (400 Bad Request):
{
  "message": "Name, canvas, and parameters are required"
}
```

11.5.3 PUT Request Example (Update Existing Diagram)

```
Request:
PUT /api/data/diagrams/{diagramId}
```

```
Authorization: Bearer <accessToken>
Content-Type: application/json

Body:
{
  "name": "UpdatedDiagram",
  "canvas": "{ \"nodes\": [...], \"edges\": [...], ... }",
  "parameters": "{ ... }",
  "updatedSnapshotData": { ... }
}

Response (200 OK):
{
  "message": "Diagram updated successfully",
  "diagram": {
    "id": "diag5678",
    "name": "UpdatedDiagram",
    "canvas": "...",
    "parameters": "..."
  }
}
```

A 403 Forbidden is returned if the user does not own the diagram.

11.5.4 DELETE Request Example (Remove a Refresh Token)

```
Request:
DELETE /api/auth/logout
Authorization: Bearer <refreshToken>

Response (200 OK):
{
  "message": "Logged out successfully"
}
```

The server deletes the associated refresh token from the database. If the token is invalid, a 403 error will be returned.

12 Timeline

The timeline for the development of each of the modules is displayed in the below table which includes module title, responsible developer(s) and the timeline in which the module was

Module Name	Responsible Developer(s)	Timeline
System Data Module (M6)	Dhrumil Pandya	Nov 2024 - Dec 2024
User Data Module (M7)	Kishor Padya	Nov 2024 - Dec 2024
Canvas Module (M5)	Madhu Sivapragasam, Rasik Pokharel	Nov 2024 - Dec 2024
Dashboard Module (M8)	Rasik Pokharel	Dec 2024 - Jan 2025
Authentication module (M2)	Kishor Pandya	Dec 2024 - Jan 2025
Save Module (M3)	Madhu Sivapragasam	Dec 2024 - Jan 2025
Restore Module (M3)	Dhrumil Pandya	Dec 2024 - Jan 2025

Table 4: Development Timelome

developed. The hardware hiding module (M1) is not being implemented and the simulation module (M9) is provided by the project sponsor and so they are not included in the timeline.

The month of February will be reserved for module testing where the developers responsible for each module will test and verify the performance of their respective modules. This will be done through manual testing as well as automated test suites. Any errors or bugs will also be fixed during this time period.

13 Appendix

This section includes the screenshots of the app's user interface, along with descriptive titles for each screen.

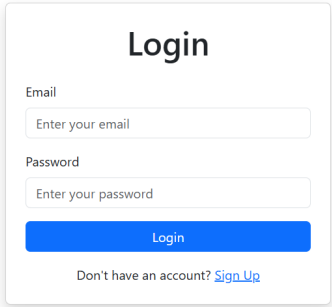

Figure 2: Login Page

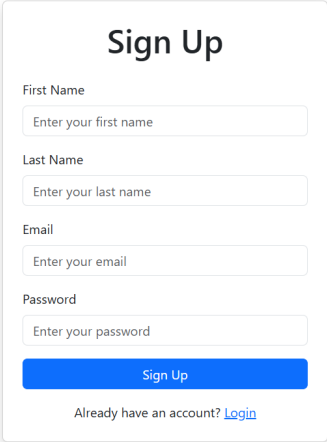

Figure 3: Sign-up Page

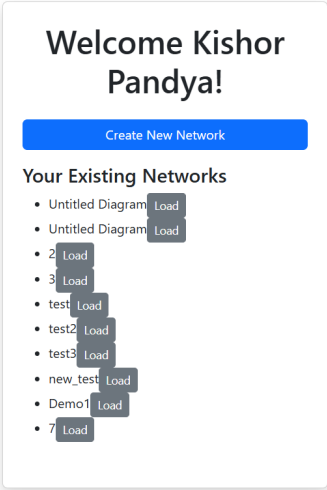

Figure 4: Dashboard Page

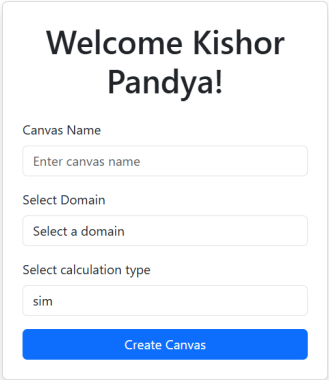

Figure 5: Dashboard Page, when create is clicked

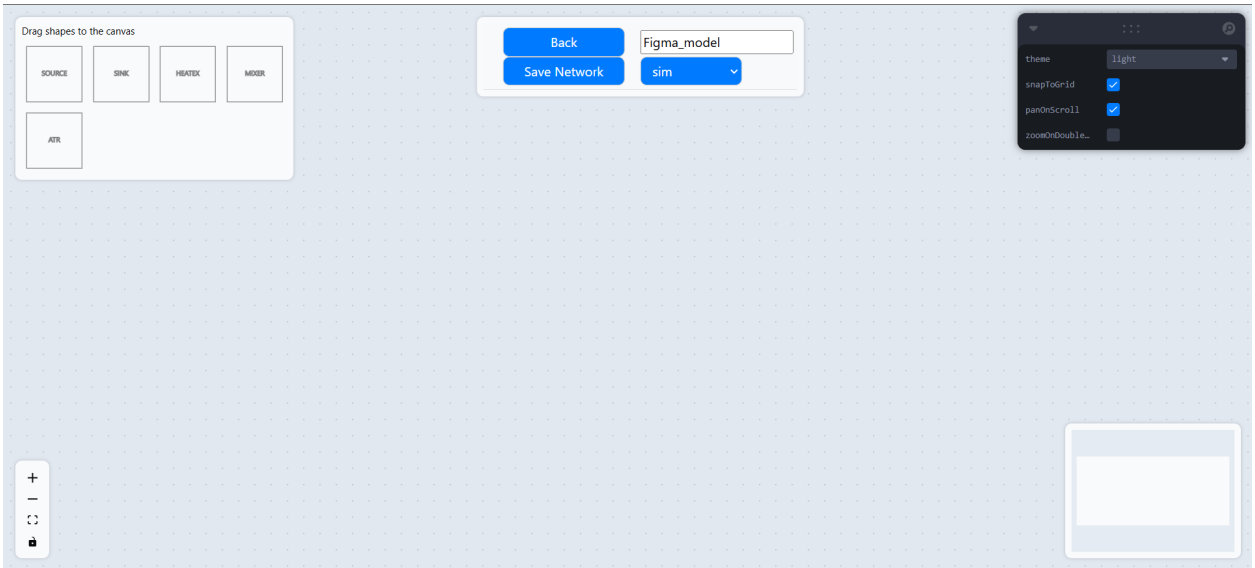

Figure 6: Canvas Page

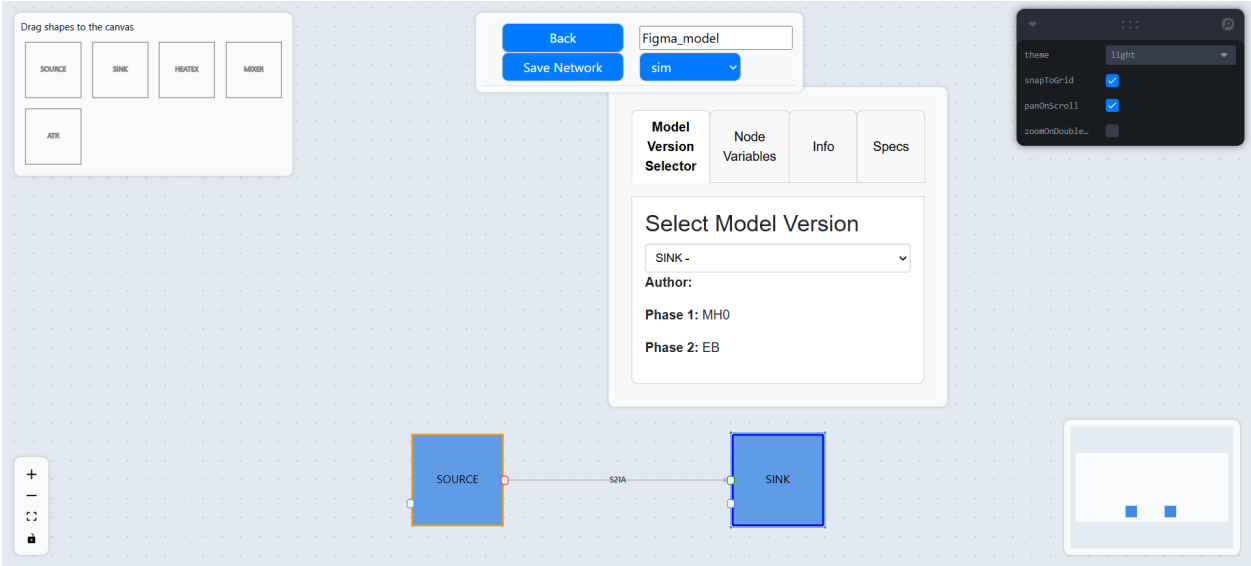

Figure 7: Canvas Page when in use

References

David L. Parnas. On the criteria to be used in decomposing systems into modules. *Comm. ACM*, 15(2):1053–1058, December 1972.

D.L. Parnas, P.C. Clement, and D. M. Weiss. The modular structure of complex systems. In *International Conference on Software Engineering*, pages 408–419, 1984.